

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ...

BÁO CÁO ĐỒ ÁN

**XÂY DỰNG HỆ THỐNG
PHÁT HIỆN BÀI BÁO TIẾNG VIỆT DO AI
TẠO RA**

Sinh viên: Nguyễn Hồ Giang Long
MSSV: 2351010115
Giảng viên hướng dẫn: Hồ Hường Thiên

TP. Hồ Chí Minh, 2026

LỜI CẢM ƠN

Nội dung lời cảm ơn của sinh viên được trình bày tại đây.

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Mục lục

Danh sách hình vẽ

Danh sách bảng

DANH MỤC TỪ VIẾT TẮT

AI	Artificial Intelligence - Trí tuệ nhân tạo
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
LLM	Large Language Model
XAI	Explainable Artificial Intelligence

MỞ ĐẦU

Lý do chọn đề tài: Trình bày sự phát triển của nội dung do AI tạo ra và nhu cầu phát hiện văn bản AI tiếng Việt.

Mục tiêu: Xây dựng mô hình phân loại văn bản Human/AI và tích hợp mô hình vào một hệ thống web có khả năng giải thích kết quả.

Đối tượng và phạm vi: Văn bản tiếng Việt, ưu tiên nội dung IT/news; hệ thống gồm pipeline dữ liệu, mô hình BamiBERT, API và giao diện web.

Phương pháp: Thu thập dữ liệu, tiền xử lý, fine-tuning, đánh giá thực nghiệm và triển khai ứng dụng.

Bố cục báo cáo: Báo cáo gồm 4 chương nội dung, từ cơ sở lý thuyết và dữ liệu đến đánh giá mô hình và triển khai hệ thống.

Chương 1

GIỚI THIỆU ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong bối cảnh hiện nay, sự bùng nổ và phát triển một cách chóng mặt của các mô hình ngôn ngữ lớn (Large Language Models - LLM) như ChatGPT, Claude, Gemini,... đã và đang làm thay đổi trong việc tạo ra và tiếp cận thông tin. Các mô hình này có khả năng tạo ra những bài báo, văn bản hay tin tức một cách nhanh chóng và cách diễn đạt ngày càng tự nhiên, mạch lạc hơn, tức là ngày càng giống như con người viết. Điều này vô tình dẫn đến việc phân biệt nội dung giữa con người hay AI tạo ra ngày càng khó hơn. Điều này làm tăng nhu cầu phát triển và gia tăng các phương pháp, hệ thống có khả năng phát hiện nội dung do AI tạo ra.

Tuy nhiên, hiện nay, có một số nhóm người lợi dụng các mô hình ngôn ngữ lớn đó để tạo ra các bài báo, tin tức nhằm phát tán thông tin sai lệch, gây hoang mang dư luận, và sử dụng các bài báo từ nguồn khác để tạo ra các bài báo mới bằng cách viết, diễn đạt lại nội dung, lấy chất xám của người khác để tạo ra các sản phẩm cho riêng mình với mục đích khác. Ngoài ra, phần lớn các nghiên cứu và công cụ phát hiện văn bản do AI tạo ra hiện nay chỉ tập trung nhiều vào tiếng Anh, trong khi đó, các bài báo tiếng Việt vẫn chưa được quan tâm đúng mức.

Để góp phần giải quyết các vấn đề trên, em đã lựa chọn đề tài **“Phát hiện văn bản do AI tạo ra và viết lại”**, tập trung vào các bài báo, tin tức tiếng Việt về lĩnh vực công nghệ thông tin (IT), các thiết bị công nghệ hiện đại, AI,... Lý do chọn lĩnh vực này là vì em nghĩ đây là một lĩnh vực có nhiều thông tin, kiến thức mới được cập nhật liên tục và phát triển rất nhanh, có sự thay đổi thuật ngữ, từ vựng và cách diễn đạt liên tục, tạo ra một thách thức nhất định đối với bài toán phát hiện các văn bản do AI tạo ra. Qua đó, góp phần hỗ trợ quá trình đánh giá và kiểm chứng nguồn gốc thông tin trên môi trường số.

1.2 Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là xây dựng một hệ thống có khả năng phát hiện các bài báo, văn bản do AI tạo ra hay người viết dựa trên kiến trúc mô hình BaniBert, đồng thời cũng đánh giá khả năng nhận diện nội dung tạo ra hay là viết lại bằng AI. Bên cạnh khả năng phân loại, hệ thống còn hỗ trợ khả năng xác định mức độ đóng góp từng từ dựa vào kết quả dự đoán bằng thuật toán diễn giải mô hình (Explainable AI) Integrated Gradients và khả năng diễn giải kết quả bằng ngôn ngữ tự nhiên thông qua API của Gemini. Hệ thống được triển khai dưới dạng web app, một giao diện tương tác người dùng thân thiện được thiết kế để người

dùng có thể tương tác với hệ thống một cách dễ dàng.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu bao gồm:

Các bài báo do con người viết và AI tạo ra, viết lại: Các bài báo, tin tức chuyên về lĩnh vực công nghệ thông tin, AI, công nghệ hiện đại,...được thu thập từ các nguồn uy tín, rõ ràng.

Các phương pháp và mô hình học sâu cho bài toán phân loại văn bản: Nghiên cứu các mô hình học sâu xử lý các đặc trưng ngôn ngữ và phương pháp học biểu diễn văn bản tiếng Việt nhằm phân biệt nội dung con người hay AI tạo ra, viết lại.

Phương pháp diễn giải mô hình (Explainable AI): Nghiên cứu các phương pháp nhằm diễn giải kết quả dự đoán của mô hình học sâu, giải thích được mức độ đóng góp từng từ cho kết quả dự đoán.

Sử dụng API của Gemini: Nghiên cứu khả năng diễn giải kết quả dự đoán bằng ngôn ngữ tự nhiên với một số ràng buộc nhất định.

Triển khai hệ thống web app: Xây dựng một hệ thống đơn giản, dễ sử dụng, dữ liệu được lưu trữ bằng SQLite, triển khai trên Docker.

1.4 Bố cục đề tài

Chương 1: Giới thiệu đề tài

Chương 2: Các công nghệ và cơ sở lý thuyết

Chương 3: Dữ liệu, phương pháp xây dựng hệ thống

Chương 4: Thực nghiệm và đánh giá hệ thống

Chương 5: Kết luận và phương hướng phát triển hệ thống trong tương lai

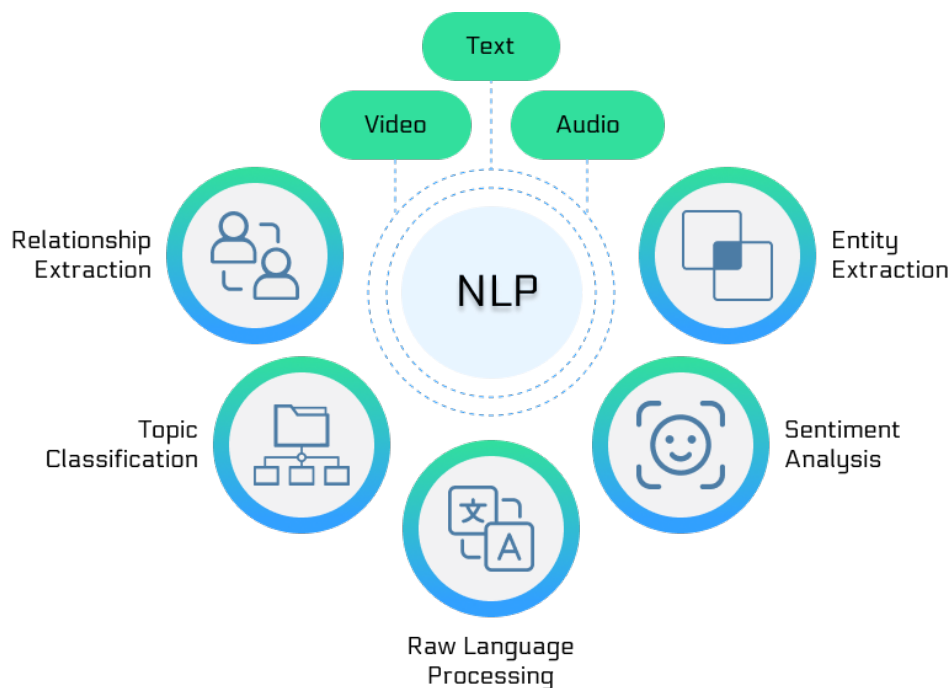
Chương 2

CÁC CÔNG NGHỆ VÀ CƠ SỞ LÝ THUYẾT

2.1 Xử lý ngôn ngữ tự nhiên

2.1.1 Vai trò của xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP) là một lĩnh vực của trí tuệ nhân tạo (Artificial Intelligence - AI) tập trung vào việc phân tích và xử lý các bài toán liên quan tới ngôn ngữ tự nhiên giúp cho máy tính có thể hiểu, phân tích và khai thác thông tin từ chính các bài toán đó.



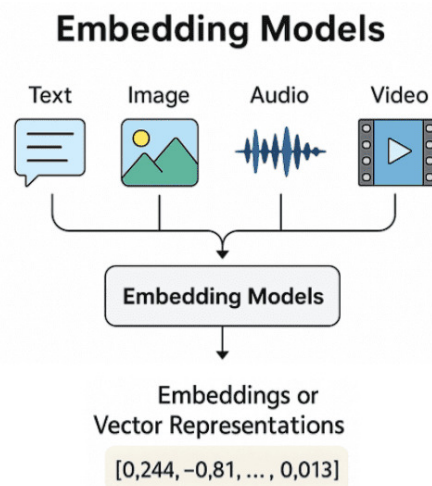
Hình 1: Xử lý ngôn ngữ tự nhiên - Natural Language Processing

Khác với dữ liệu số hoặc dữ liệu có cấu trúc, dữ liệu văn bản ngôn ngữ tự nhiên có nhiều đặc điểm phức tạp, chẳng hạn như ngữ nghĩa, ngữ cảnh, cách sử dụng từ và cấu trúc câu. Do đó, NLP đóng vai trò quan trọng trong việc chuyển đổi văn bản thành dạng dữ liệu có thể đưa vào các mô hình học máy và học sâu xử lý.

2.1.2 Biểu diễn văn bản và embedding

Để cho máy tính có thể hiểu được văn bản, nội dung của văn bản đó sẽ được chuyển đổi sang dạng biểu diễn số. Một trong những phương pháp phổ biến để giải quyết được yêu cầu đó là chuyển đổi văn bản dưới dạng vector số, trong đó mỗi vector đều mang, chứa đựng các thông tin bao gồm: ý nghĩa, ngôn ngữ, cấu trúc của từ, cụm từ và câu. Các phương pháp truyền thống như Bag-of-Words hoặc TF-IDF chủ yếu biểu diễn vector dựa vào sự xuất hiện và tần suất của từ, vẫn chưa đủ khả năng để phản ánh đầy đủ mối quan hệ ngữ nghĩa và ngữ cảnh của từ trong câu.

Embedding là một kỹ thuật chuyển đổi dữ liệu văn bản thô như từ ngữ, câu văn, hình ảnh hay là âm thanh sang một vector trong không gian đa chiều. Mỗi đơn vị mà có ý nghĩa tương đồng thì các vector của chúng sẽ được biểu diễn gần nhau trong không gian đa chiều đó. Nhờ vậy, embedding có thể giúp cho mô hình máy có thể hiểu và khai thác tốt hơn những đặc điểm ngữ nghĩa thay vì chỉ dựa trên sự xuất hiện riêng lẻ của từng từ.



Hình 2: Biểu diễn dữ liệu thô dưới dạng vector embedding

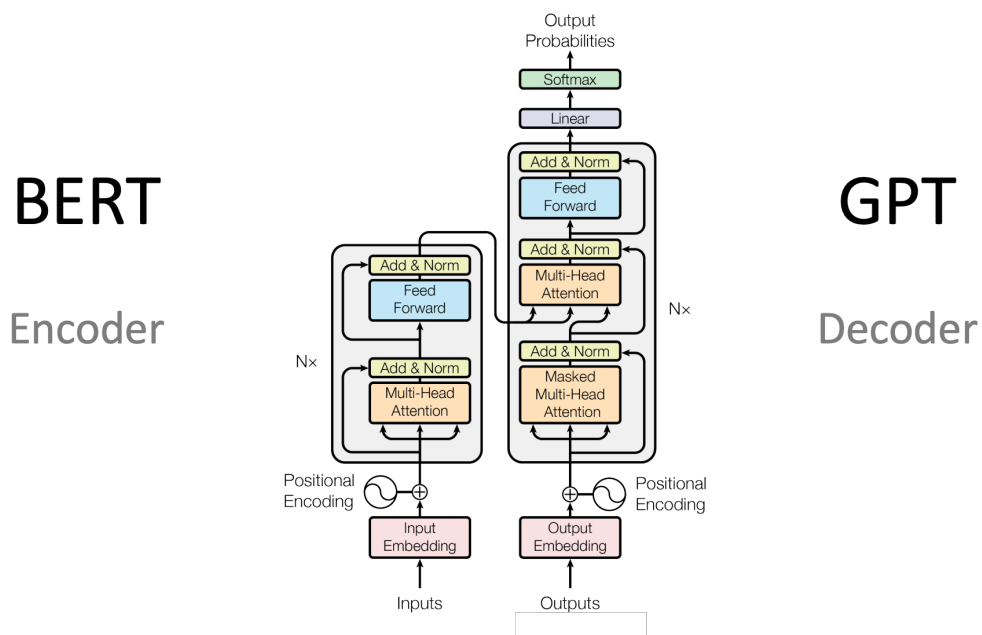
Trong các hệ thống xử lý văn bản hiện tại, embedding là thành phần rất quan trọng trong việc hỗ trợ chuyển đổi dữ liệu dạng ngôn ngữ tự nhiên sang dữ liệu số, hay là vector để máy tính có thể hiểu, xử lý và tính toán được. Các vector embedding đó sẽ được đưa vào các mô hình học để thực hiện một số tác vụ như phân loại văn bản, tìm kiếm thông tin, phát hiện tương đồng và phân tích ngữ nghĩa.

Đối với bài toán phân loại văn bản AI tạo ra, việc biểu diễn văn bản dưới dạng dữ liệu số có thể giúp cho các mô hình khai thác tốt đặc trưng từ dữ liệu đầu vào, tăng khả năng nhận diện được các văn bản do người viết hay do AI tạo ra. Tuy vậy, chất lượng biểu diễn các đặc trưng cũng góp phần cực kỳ quan trọng, ảnh hưởng trực tiếp đến kết quả đầu ra trong việc phân loại và đánh giá của mô hình.

2.2 Mô hình Transformer và BERT

2.2.1 Kiến trúc Transformer

Transformer là một kiến trúc mạng nơ-ron được thiết kế nhằm mục đích xử lý dữ liệu tuần tự, đặc biệt hiệu quả đối với các bài toán xử lý ngôn ngữ tự nhiên. Khác với các kiến trúc tuần tự như Recurrent Neural Network (RNN) và Long Short-Term Memory (LSTM), Transformer có khả năng xử lý các token trong chuỗi một cách đồng thời, không theo tuần tự. Có nghĩa là, mô hình sẽ tính toán mức độ liên quan của một token so với các token khác tồn tại trong chuỗi. Nhờ đó, một token sẽ mang ý nghĩa phù hợp với ngữ cảnh, tức là phù hợp với ý nghĩa của câu đó mà không phải phụ thuộc vào mỗi bản thân của token đó.



Hình 3: Kiến trúc mô hình Transformer

Cơ chế mà xác định mức độ quan trọng của từng token dựa vào các token lân cận có trong chuỗi được gọi là cơ chế tự chú ý (Self-Attention). Qua đó, Transformer có khả năng mô hình hóa các mối quan hệ phụ thuộc xa trong văn bản, tức là một token cần thêm thông tin từ một token rất xa trong văn bản, một đặc điểm quan trọng đối với các bài toán xử lý ngôn ngữ tự nhiên.

Transformer gồm tất cả là hai thành phần, Encoder và Decoder. Phần đầu là Encoder, dữ liệu đầu vào sẽ được chia thành các token (Tokenization) rồi sau đó sẽ embedding thành các vector. Tiếp theo, các vector đó sẽ được qua một lớp, gọi là Positional Embedding, lớp này sẽ có trách nhiệm embedding thành các vector có tính đại diện vị trí của từ trong chuỗi dữ liệu đó, ví dụ như: "I love AI" và "AI love I" thì mô hình nhìn có vẻ giống nhau nhưng thực tế thì không phải vậy, thế nên là lớp Positional Embedding góp phần rất là quan trọng. Qua đó, ta sẽ có được một vector vừa có thông tin về ngữ nghĩa và vị trí của từ đó trong câu. Sau đó, các vector embedding sẽ được đưa vào lớp tự chú ý (Self-Attention) như đã nói ở trên, mỗi từ sẽ được ánh xạ thành ba thành phần: Query (Q) - từ này đang tìm kiếm cái gì, Key (K) - từ này yêu cầu cái gì, và Value (V) - thông tin thực sự mà từ này mang lại cái gì. Mỗi từ sẽ sử dụng Q để tính tích vô hướng so với các K của các từ còn lại, giá trị vừa tính được sẽ

được chuẩn hóa thành các khoảng giá trị có thể xử lý được rồi sẽ đưa vào hàm Softmax để đưa thành các giá trị xác suất $[0, 1]$.

Sau đó, dữ liệu sẽ đi qua lớp đa đầu tự chú ý (Multi-Head Self Attention), lớp này đảm bảo cho việc chú ý sẽ chạy song song, mỗi đầu tập trung vào một công việc riêng, ví dụ như một đầu tập trung vào mối quan hệ chủ ngữ - động từ, một đầu tập trung vào tính từ đó mô tả danh từ nào. Tiếp theo, dữ liệu sẽ được đưa vào lớp mạng nơ-ron truyền thẳng (Feed-Forward Network), lớp này sẽ hỗ trợ cho việc đào sâu thông tin của từng từ hơn, ý nghĩa chính xác hơn và loại bỏ nhiễu kết hợp với các lớp Add & Norm để ổn định quá trình học hơn. Phần Decoder cũng có cấu trúc tương tự như Encoder nhưng thêm một lớp là Masked Multi-Head Attention, là lớp che đi các token phía sau của token đang xét để nó không biết trước thông tin. Điều này tránh rò rỉ thông tin trong quá trình học. Các khối Encoder và Decoder có thể lặp lại nhiều lần.

2.2.2 BERT và BamiBERT

BERT (Bidirectional Encoder Representations from Transformers) là một mô hình ngôn ngữ tiền huấn luyện được phát triển dựa trên kiến trúc Transformer Encoder. Nhờ vậy, mỗi token có thể khai thác thông tin từ hai chiều, phía trước và phía sau của token đang xét nên mô hình có khả năng nắm bắt tốt mối quan hệ các từ dựa trên ngữ cảnh của câu.

BERT được tiền huấn luyện trên một bộ lượng lớn văn bản thông qua nhiều nhiệm vụ học ngôn ngữ khác nhau, trong đó phổ biến nhất là mô hình ngôn ngữ che giấu (Masked Language Modeling - MLM). Cụ thể, trong nhiệm vụ này thì một số token trong câu sẽ bị che lại (masked) và mô hình có nhiệm vụ phải dự đoán các token bị che đó dựa vào các token còn lại. Quá trình này hỗ trợ cho mô hình BERT học được các đặc trưng của từ vựng, ngữ nghĩa và mối quan hệ giữa các thành phần trong văn bản.

Sau giai đoạn tiền huấn luyện, BERT có thể được fine-tune, tức là tinh chỉnh, huấn luyện trên tập dữ liệu của mình cho nhiều tác vụ khác nhau. Cụ thể trong bài toán phân loại văn bản, các biểu diễn thu được, tức là các vector embedding từ dữ liệu ban đầu từ BERT sẽ được đưa vào một lớp phân loại để dự đoán nhãn tương ứng. Cách tiếp cận này tận dụng kiến thức mà mô hình đã học được trong quá trình tiền huấn luyện thay vì xây dựng lại từ đầu.

BamiBERT là một mô hình được xây dựng dựa trên kiến trúc BERT, được huấn luyện trên phần lớn các tập dữ liệu tiếng Việt với nhiều tác vụ khác nhau, bao gồm suy luận ngôn ngữ, nhận dạng thực thể, phân tích cảm xúc, phân loại chủ đề, phát hiện spam và phân tích khía cạnh.

Dataset	Train.	Valid.	Test
ViNLI	24,376	3,009	2,991
PhoNER_COVID19	5,027	2,000	3,000
UIT-VSFC (Sentiment)	11,426	1,583	3,166
UIT-VSFC (Topic)	11,426	1,583	3,166
ViSpamReviews	14,306	1,590	3,974
UIT-ViSFD	7,786	1,112	2,224
UIT-ABSA (Hotel)	7,180	795	2,030
UIT-ABSA (Restaurant)	7,028	771	1,938

Table 1: Statistics of 8 experimental datasets.

Hình 4: Các tập dữ liệu dùng để huấn luyện BamiBERT

Trong hệ thống, BamiBERT đóng vai trò là một mô hình biểu diễn các đặc trưng và sau đó biểu diễn này được sử dụng cho nhiệm vụ phân loại các lớp để đưa ra dự đoán. Việc sử dụng mô hình tiền huấn luyện giúp hệ thống tận dụng được các đặc trưng ngôn ngữ đã học trước đó và phù hợp hóa với bài toán phát hiện nội dung AI.

Dataset	Metric	BamiBERT	ViDeBERTa	ViSoBERT	XLM-RoBERTa	PhoBERT
ViNLI (4-label)	Accuracy	81.01	61.08	67.70	76.83 [†]	<u>78.00[†]</u>
	F ₁	81.15	60.71	67.82	77.01 [†]	<u>78.05[†]</u>
PhoNER_COVID19	F ₁	94.90	<u>94.50[†]</u>	92.90	92.50 [†]	94.20 [†]
UIT-VSFC (Sentiment)	Accuracy	<u>93.86</u>	87.86	93.15	93.56	94.10
	F ₁	83.41	70.74	81.49	82.20	<u>83.27</u>
UIT-VSFC (Topic)	Accuracy	89.34	83.94	88.80	89.18	<u>89.24</u>
	F ₁	79.90	66.49	<u>79.86</u>	79.56	79.80
ViSpamReviews	Accuracy	<u>90.76</u>	86.21	90.99[†]	90.16 [†]	89.83 [†]
	F ₁	<u>78.20</u>	67.04	79.06[†]	76.55 [†]	76.18 [†]
UIT-ViSFD	F ₁ (Detection)	89.14	75.53	<u>88.63</u>	82.73	86.03
	F ₁ (Classification)	84.24	64.20	<u>83.55</u>	65.81	78.76
UIT-ABSA (Hotel)	F ₁ (Detection)	79.99	72.05	<u>79.41</u>	77.70 [†]	79.16 [†]
	F ₁ (Classification)	72.65	62.97	74.24	71.23 [†]	<u>73.73[†]</u>
UIT-ABSA (Restaurant)	F ₁ (Detection)	88.01	73.56	<u>86.86</u>	82.18 [†]	86.53 [†]
	F ₁ (Classification)	74.89	63.78	<u>73.87</u>	71.58 [†]	73.52 [†]

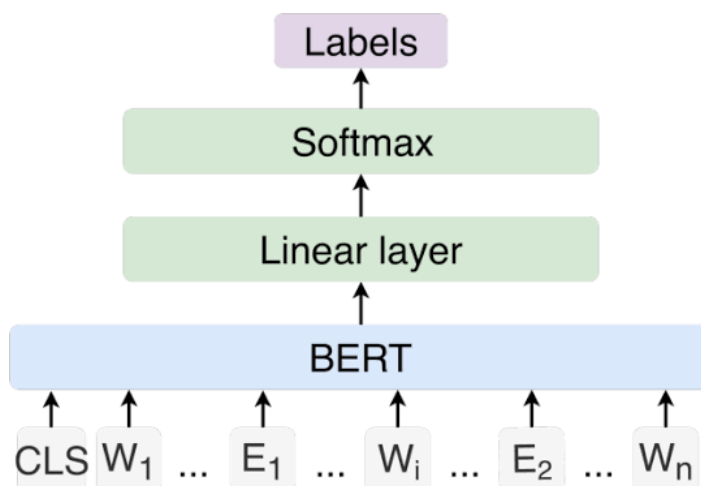
Table 2: Results of pre-trained "base"-architecture models. [†] denotes results extracted from previous works.

Hình 5: Kết quả dự đoán của BamiBERT so với các mô hình BERT khác

Lý do lựa chọn BamiBERT làm mô hình trích xuất, phân loại cho đề án này là vì độ chính xác của mô hình tốt hơn hẳn so với các mô hình BERT trước dựa vào bảng kết quả (Hình 5). Tuy nhiên, có thể trong một số bài toán khác, hay cụ thể là bài toán phân loại văn bản AI này thì một số mô hình BERT khác có khả năng xử lý tốt và đưa ra độ chính xác cao hơn. Do đó, bảng kết quả trên chỉ là thước đo cơ sở để lựa chọn mô hình BamiBERT nhưng về vấn đề xử lý hay độ chính xác cao hơn thì có thể các mô hình BERT khác tốt hơn so với BamiBERT.

2.2.3 Token [CLS] và vector đại diện cho câu

Trong kiến trúc BERT, token [CLS] (Classification) là một token đặc biệt được thêm vào vị trí đầu tiên của chuỗi đầu vào. Token này có nhiệm vụ tổng hợp toàn bộ thông tin của chuỗi đang xử lý thông qua các lớp Encoder của Transformer. Trong quá trình xử lý chuỗi, cơ chế Self-Attention sẽ cho phép token [CLS] tương tác với tất cả các token còn lại trong chuỗi, nhờ đó mà thu được ngữ cảnh của toàn bộ văn bản trong chuỗi.



Hình 6: Token [CLS] trong chuỗi

Sau khi chuỗi đầu vào đi qua các lớp Encoder, mỗi token sẽ có một vector embedding tương ứng. Tuy nhiên, token [CLS] có một vector đặc biệt, là vector đại diện cho toàn bộ câu trong chuỗi. Vector này chứa toàn bộ thông tin ngữ cảnh của chuỗi đầu vào và được sử dụng cho các bài toán phân loại. Cụ thể, trong bài toán của đề án này, vector biểu diễn của token [CLS] từ BamiBERT được sử dụng làm đầu vào cho Classification Head, sau đó vector này sẽ được chuyển đổi thành các giá trị tương ứng cho hai lớp Human và AI cho bài toán, cụ thể các giá trị đó là giá trị logit.

2.2.4 Classification head

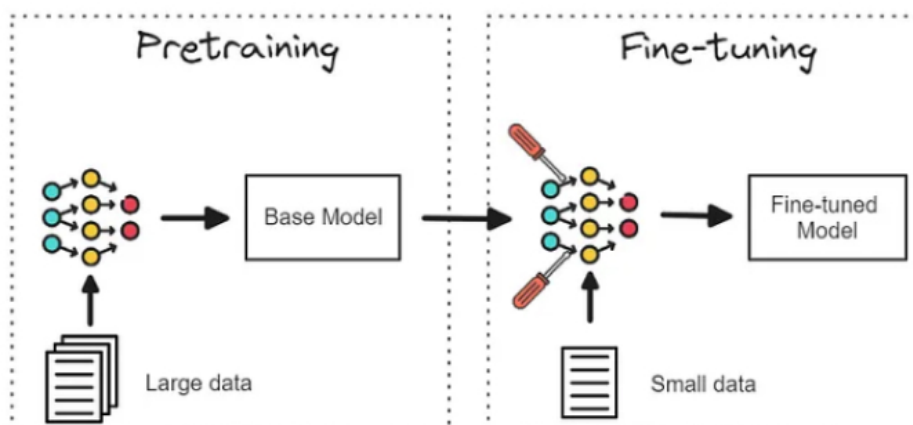
Classification head là một lớp của mô hình BERT, cho phép nhận đầu vào là vector đại diện của token [CLS] để thực hiện nhiệm vụ phân loại. Thành phần này là cầu nối giữa mô hình tiền huấn luyện và nhiệm vụ phân loại cụ thể.

Trong bài toán phân loại của đề án, lớp classification head nhận đầu vào sẽ nhận vector đại diện của BamiBERT làm đầu vào. Sau đó, vector này sẽ được đưa qua một lớp gọi là lớp kết nối đầy đủ (Fully Connected Layer) để ánh xạ từ không gian nhiều chiều của BamiBERT sang không gian nhẵn của bài toán phân loại của đề án, tức là đề án chỉ gồm có hai lớp, AI và Human nên không gian của bài toán này chỉ có hai chiều và giá trị là logit. Tiếp theo, các giá trị này sẽ được trực tiếp đưa vào hàm Softmax để chuyển đổi thành các giá trị xác suất làm kết quả dự đoán.

2.2.5 Fine-tune mô hình tiền huấn luyện

Fine-tune là quá trình tiếp tục huấn luyện mô hình đã được tiền huấn luyện trên một tập dữ liệu cụ thể, một bài toán mà ta đang giải quyết. Tức là thay vì khởi tạo các tham số lại từ đầu

thì quá trình này sẽ tận dụng các tham số mà mô hình này đã học được trong quá trình tiền huấn luyện, rồi điều chỉnh sao cho phù hợp với mục tiêu của bài toán.



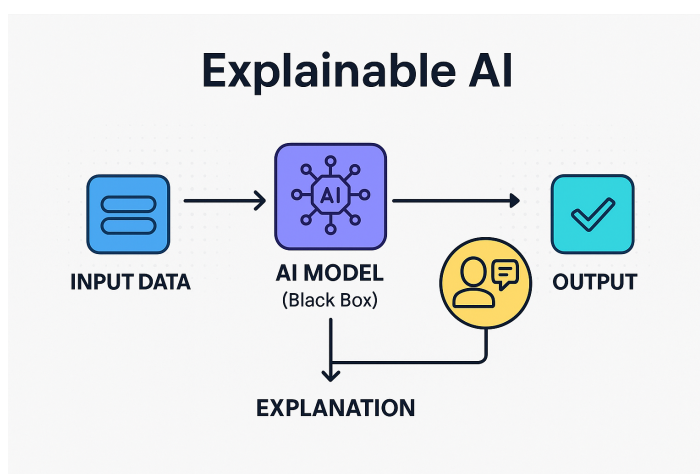
Hình 7: Quá trình fine-tuning

Đối với mô hình BamiBERT được sử dụng trong đồ án, giai đoạn tiền huấn luyện ban đầu đã phần lớn giúp cho mô hình đã học được các dữ liệu tiếng Việt. Tuy nhiên, mô hình này vẫn chưa được huấn luyện tối ưu cho bài toán phân loại, cụ thể là phân loại tin tức AI hay là Human. Do đó, đồ án thực hiện fine-tune mô hình trên tập dữ liệu của bài toán, đồng thời kết hợp thêm lớp Classification Head để thực hiện phân loại văn bản đó là người hay AI tạo ra.

2.3 Thuật toán diễn giải mô hình (Explainable AI - XAI)

2.3.1 Khái niệm Explainable AI

Thuật toán diễn giải mô hình (Explainable AI) là một nhóm, tập hợp các phương pháp và kỹ thuật giúp cho con người cái cách mà một mô hình AI đưa ra dự đoán. Trong mô hình học sâu, thông thường thì quá trình dự đoán rất khó giải thích trực tiếp vì mô hình chứa rất nhiều tham số và thực hiện các thuật toán, quá trình tính toán cực kỳ phức tạp. XAI cung cấp phương pháp, thuật toán để giải thích mối quan hệ giữa dữ liệu đầu vào so với kết quả đầu ra của mô hình, điều này góp phần lớn cho việc giải thích minh bạch, dễ hiểu cho người dùng.



Hình 8: Thuật toán diễn giải mô hình - Explainable AI

Trong bài toán phân loại văn bản, thuật toán XAI được sử dụng để xác định những từ, token nào trực tiếp góp phần cho kết quả dự đoán, tức là mức độ ảnh hưởng của từ đó đối với kết quả mà mô hình dự đoán được. Ví dụ, trong đồ án này, khi mô hình dự đoán văn bản đó thuộc về lớp AI thì thuật toán sẽ trực tiếp xác định từ hay token nào trong văn bản đó đóng góp đáng kể cho kết quả dự đoán. Nhờ vậy, kết quả của mô hình không những thể hiện dưới dạng nhãn dự đoán mà còn có thể cung cấp thông tin lý do vì sao lại đưa ra kết quả dự đoán đó. Nhờ đó, XAI hỗ trợ rất tốt trong việc phân tích lỗi và đánh giá các đặc trưng mô hình đã sử dụng trong quá trình phân loại.

2.3.2 Thuật toán Integrated Gradients

Thuật toán Integrated Gradients (IG) là một thuật toán XAI, sử dụng để xác định mức độ đóng góp của từng token so với kết quả dự đoán của mô hình. Phương pháp này sử dụng đạo hàm (gradient) của mô hình để phân tích mức độ ảnh hưởng của dữ liệu đầu vào đến đầu ra, tức là xem đầu ra của mô hình như thế nào khi thay đổi nhỏ đầu vào.

Ý tưởng chính của thuật toán này là xác định mức độ ảnh hưởng của từng đặc trưng dữ liệu đầu vào so với kết quả đầu ra của mô hình. Cách thức hoạt động của thuật toán như sau, đầu tiên IG sẽ bắt đầu từ đầu vào tham chiếu, tức là điểm xuất phát để thuật toán so với câu thực tế (baselines) và thay đổi dần cho đến dữ liệu thực tế. Tại mỗi bước, IG sẽ theo dõi sự thay đổi trong kết quả dự đoán của mô hình khi dữ liệu đầu vào thay đổi một chút. Sau khi thực hiện qua tất cả các bước thì thuật toán này sẽ tổng hợp lại các kết quả để xem đặc trưng nào có ảnh hưởng ít nhiều so với kết quả dự đoán.

$$IG_i(x) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial F_i(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

Hình 9: Công thức tích phân gradient tích hợp của thuật toán Integrated Gradients

Trong đó:

- x : dữ liệu thực tế cần giải thích.
- x' : baseline.
- F : đầu ra của mô hình.
- x_i : đặc trưng thứ i .
- $IG_i(x)$: mức độ đóng góp của đặc trưng thứ i .

Trong công thức trên, $x_i - x'_i$ thể hiện sự khác biệt giữa đặc trưng thứ i của dữ liệu thực tế so với đặc trưng tương ứng của dữ liệu baseline mà ta đang xét. Ngoài ra, phần đạo hàm $\frac{\partial F_i}{\partial x_i}$ cho biết đầu ra của mô hình thay đổi như thế nào khi mà thuật toán thay đổi nhỏ đặc trưng thứ i . Thay vì tính toán mức độ ảnh hưởng tại một thời điểm, thuật toán IG sẽ tính toán mức độ thay đổi này tại nhiều điểm trong quá trình đi từ baseline cho đến dữ liệu thực tế và tổng hợp các giá trị vừa tính được qua phép tích phân với α chạy từ 0 đến 1. Trong đó, 0 ở đây tức là vẫn đang ở baseline, 0.5 thì tức là đang ở giữa baseline và dữ liệu thực tế và 1 đang ở ngay dữ liệu thực tế. Kết quả $IG_i(x)$ cho biết mức độ đóng góp đặc trưng thứ i đối với kết

qua dự đoán, giá trị càng lớn thì càng ảnh hưởng lớn đối với kết quả dự đoán. Nếu giá trị là dương, thì đặc trưng đang có xu hướng nghiêng ủng hộ phía lớp được dự đoán, còn giá trị âm thì ngược lại.

Ví dụ như trong đề án phân loại văn bản AI này, các đặc trưng đầu vào đã được biểu diễn dưới dạng embedding của các token trong chuỗi văn bản. Do đó, thuật toán IG sẽ được sử dụng để xác định mức độ đóng góp của từng token đối với kết quả dự đoán của mô hình BamiBERT, những token có mức độ đóng góp lớn thì có ảnh hưởng lớn đến kết quả do mô hình BamiBERT dự đoán, mức độ ảnh hưởng đó được tính thành một điểm được gọi là điểm đóng góp (Attribution Score).

2.3.3 Attribution trong bài toán phân loại văn bản

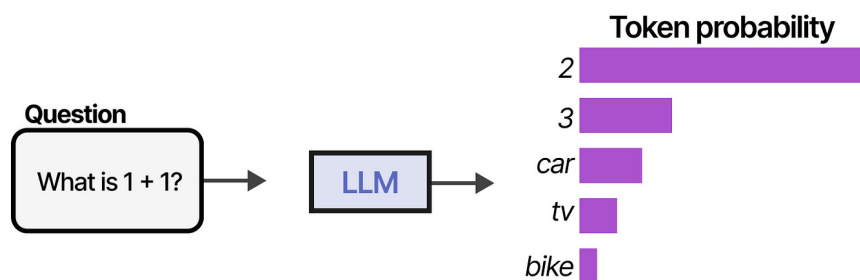
Attribution tức là mức độ đóng góp của các đặc trưng đầu vào đối với một kết quả cụ thể mà mô hình tạo ra. Cụ thể, đối với mô hình BamiBERT thì các đặc trưng này tương ứng với từng token trong chuỗi văn bản đang xét. Thông qua thuật toán IG, mỗi token sẽ được gán giá trị điểm đóng góp (Attribution Score) để biết được mức độ ảnh hưởng của token đó đối với kết quả dự đoán.

Trong bài toán phân loại văn bản AI, nếu attribution của một token có giá trị dương thì token đó có xu hướng hỗ trợ cho kết quả dự đoán của lớp đang xét, còn nếu token có giá trị âm thì ngược lại. Ngoài ra, độ lớn tuyệt đối của một token càng lớn thì token đó càng ảnh hưởng mạnh đến kết quả dự đoán của lớp mục tiêu. Nhờ vậy mà cái việc trực quan hóa khả năng diễn giải sẽ rất dễ hiểu cho người dùng.

2.4 Mô hình ngôn ngữ lớn và ứng dụng trong hệ thống

2.4.1 Mô hình ngôn ngữ lớn (Large Language Model)

Mô hình ngôn ngữ lớn (Large Language Model - LLM) là một loại mô hình AI được huấn luyện trên lượng lớn dữ liệu văn bản, có khả năng biểu diễn, đọc hiểu, xử lý và tạo ra văn bản bằng ngôn ngữ tự nhiên giống như con người, ví dụ như mô hình GPT, Gemini, Ollama,... Thông qua quá trình huấn luyện, mô hình LLM có khả năng nắm bắt, học được mối quan hệ giữa các từ, cấu trúc câu và ngữ cảnh, từ đó có thể thực hiện nhiều tác vụ như là đọc và trả lời câu hỏi, dịch, tóm tắt văn bản hay là sinh nội dung.



Hình 10: Mô hình ngôn ngữ lớn - Large Language Model (LLM)

Phần lớn các mô hình LLM đều hoạt động dựa trên kiến trúc Transformer, sử dụng cơ chế

Self-Attention để khai thác tốt ngữ cảnh trong câu. Đối với quá trình sinh văn bản, LLM sẽ dự đoán token tiếp theo bằng cách tính xác suất của các token tiếp theo dựa trên các token hiện có và lựa chọn token dựa trên phân phối xác suất. Sau đó, token sẽ được thêm vào trong câu và quá trình này sẽ được lặp lại cho tới khi câu được hoàn thiện. Ngoài ra, khác với các mô hình được xây dựng dựa trên một tác vụ cụ thể thì LLM có khả năng thực hiện nhiều nhiệm vụ khác nhau dựa yêu cầu do người dùng cung cấp dưới dạng prompt, nhờ đó mà mô hình có khả năng thực hiện nhiều nhiệm vụ khác nhau mà không cần phải huấn luyện lại mô hình từ đầu ứng với nhiệm vụ khác nhau.

2.4.2 Ứng dụng LLM trong ứng dụng giải thích kết quả

Trong các hệ thống AI hiện nay, thông thường các kết quả dự đoán của mô hình thường được biểu diễn dưới dạng nhãn, xác suất hoặc các giá trị định lượng thì sẽ gây khó hiểu cho người dùng trong việc hiểu nguyên nhân dẫn đến kết quả. Do đó, LLM thường được sử dụng bằng cách chuyển các giá trị định lượng đó sang giải thích bằng ngôn ngữ tự nhiên, nhờ vậy mà kết quả hệ thống sẽ trở nên minh bạch, dễ hiểu hơn cho người dùng.



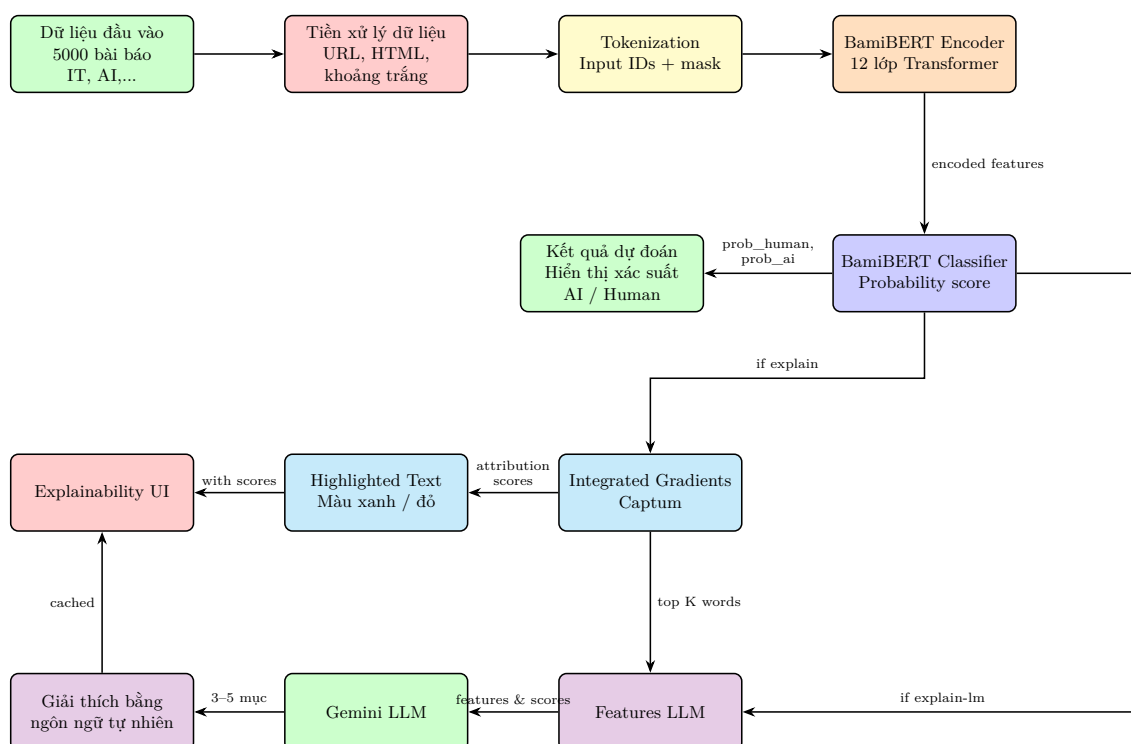
Hình 11: Mô hình ngôn ngữ lớn (Gemini)

Trong đồ án phân loại văn bản AI, IG được sử dụng để xác định mức độ ảnh hưởng, đóng góp của các token cho kết quả dự đoán của mô hình BaniBERT. Tuy nhiên, các giá trị số thường rất khó giải thích và khó hiểu cho người dùng. Do đó, trong đồ án này đã quyết định sử dụng mô hình Gemini để sử dụng và phân tích những giá trị định lượng đó, tạo ra những lời giải thích mà chỉ phụ thuộc vào các giá trị mà thuật toán, mô hình đã tính ra bằng ngôn ngữ tự nhiên nhằm giúp cho người dùng hiểu được những thành phần nào trong văn bản có ảnh hưởng đến kết quả phân loại.

Chương 3

Dữ liệu và phương pháp xây dựng hệ thống

3.1 Tổng quan hệ thống



Hình 12: Kiến trúc hệ thống

Trong hệ thống trên, dữ liệu đầu vào tập trung vào các bài báo tiếng Việt chuyên về lĩnh vực công nghệ thông tin (IT), AI hay là các thiết bị công nghệ hiện đại. Dữ liệu sau đó được tiền xử lý bằng cách loại bỏ các URL, HTML, và khoảng trắng thừa, dữ liệu đã được làm sạch sẽ được đưa vào quá trình token hóa, để ánh xạ mỗi từ sang các chỉ số đầu vào (input IDs) và mặt nạ (mask). Tiếp theo, các token này sẽ được đưa vào mô hình BamiBERT để embedding, tức là chuyển các token thành các vector đặc trưng rồi bắt đầu quá trình huấn luyện. Đầu ra của mô hình chính là xác suất dự đoán của văn bản dựa vào hai lớp, Human hay là AI.

Ngoài ra, hệ thống còn có khả năng giải thích kết quả do mô hình dự đoán bằng cách sử dụng thuật toán Integrated Gradients, thuật toán này sẽ tính điểm đóng góp cho từng token của đoạn văn bản, dựa vào điểm đóng góp đó thì hệ thống sẽ hiển thị giao diện bằng cách highlight các từ đó theo các màu tương ứng với hai lớp, cụ thể màu đỏ là AI và màu xanh là người viết, độ sáng của các màu thể hiện mức độ đóng góp, ảnh hưởng đối với kết quả dự đoán, màu càng đậm thì càng ảnh hưởng lớn đối với kết quả dự đoán.

Tiếp theo để hệ thống có khả năng giải thích bằng ngôn ngữ tự nhiên, hệ thống đã sử dụng một mô hình LLM, đó là Gemini, với sự ràng buộc yêu cầu (prompt) dựa trên các từ có điểm đóng góp cao từ thuật toán IG, và các chỉ số đo lường ví dụ như mật độ dấu câu, độ dài câu,...Sau đó câu lệnh sẽ được mô hình Gemini xử lý và trả về kết quả giải thích bằng ngôn ngữ tự nhiên, kết quả này sẽ được hiển thị trên giao diện của người dùng.

3.2 Thu thập và xây dựng dữ liệu

3.2.1 Nguồn dữ liệu

Nguồn dữ liệu được thu thập trong đồ án chủ yếu là đến từ các trang báo, bài viết tiếng Việt thuộc lĩnh vực công nghệ thông tin, AI hay các thiết bị công nghệ hiện đại, ...Việc lựa chọn một lĩnh vực cụ thể nhằm đảm bảo dữ liệu có tính thống nhất về chủ đề, đồng thời hạn chế về sự khác biệt về chủ đề có ảnh hưởng đến quá trình học của mô hình. Ngoài ra, lĩnh vực công nghệ có lượng lượng bài báo, nội dung thường xuyên đăng tải liên tục trực tuyến, thường có sự thay đổi về công nghệ, thuật ngữ chuyên môn, tên sản phẩm và tên công nghệ, ...Điều này tạo nên tính đa dạng về dữ liệu vì cách diễn đạt, cấu trúc bài viết khác nhau.

```
source
ictnews_vietnamnet    1676
quantrimang           1428
thanhnien_congnghe    236
vnexpress_sohoa        162
genk_recent            152
tuoitre_congnghe       96
Name: count, dtype: int64
```

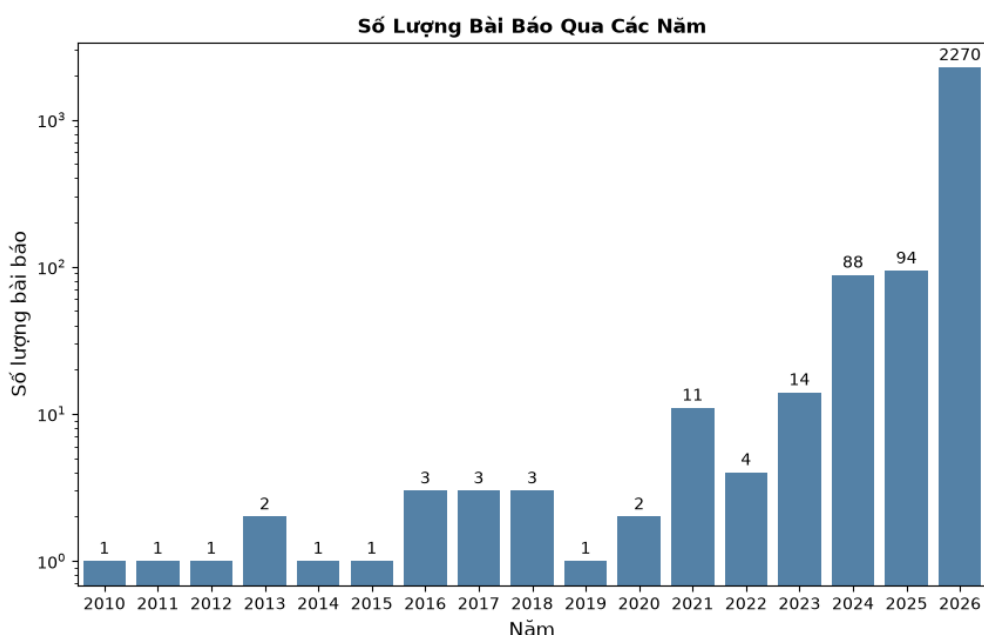
Hình 13: Các nguồn dữ liệu thu thập

Phần lớn các bài báo, tin tức được thu thập từ các nguồn uy tín, tên miền rõ ràng, đều là các trang bài báo công nghệ phổ biến ở Việt Nam, bao gồm VnExpress, GenK, Thanh Niên Công Nghệ,...Các bài báo được thu thập từ các nguồn này đều có nội dung liên quan đến lĩnh vực công nghệ thông tin, AI hay các thiết bị công nghệ hiện đại,...Các nguồn này được lựa chọn dựa trên mức độ phổ biến, ổn định và độ tin cậy của các trang tin tức Việt Nam. Nhờ vậy mà dữ liệu có nguồn gốc rõ ràng và phù hợp với phạm vi nghiên cứu.

3.2.2 Thu thập dữ liệu văn bản Human

Dữ liệu các bài báo, văn bản Human được thu thập tập trung vào các lĩnh vực công nghệ thông tin, trí tuệ nhân tạo, thiết bị công nghệ hiện đại như đã trình bày ở mục 3.2.1. Mục đích của quá trình này là để xây dựng một bộ dữ liệu bài báo, văn bản được viết và xuất bản bởi

con người, lấy làm cơ sở để mô hình học cách phân biệt nội dung giữa con người hay AI tạo ra. Quá trình thu thập dữ liệu được tự động hóa bằng cách truy cập các trang tin tức và trích xuất nội dung từ các bài báo, bài viết phù hợp. Đối với mỗi bài viết, các phần quan trọng như URL, nguồn, tên tác giả và ngày xuất bản sẽ được lưu trữ nhằm thể hiện các bài báo đó có nguồn rõ ràng, minh bạch và nội dung của bài báo đó. Các phần không liên quan đến nội dung như quảng cáo, menu hay các thành phần giao diện web sẽ được loại bỏ để đảm bảo dữ liệu thu được chỉ tập trung vào các nội dung cần thiết.



Hình 14: Các bài báo văn bản thu thập qua các năm

Như đã thấy ở Hình 14, dữ liệu được thu thập từ các bài báo, tin tức nằm trong khoảng từ năm 2010 cho đến năm 2026 và tập trung hầu hết vào năm 2026 với tổng cộng là 2.270 bài. Nguyên nhân là trong quá trình thu thập dữ liệu, các nguồn tin tức thường cung cấp số lượng bài báo mới hiện nay nhiều hơn so với các bài viết từ các năm trước, ngoài ra thì các bài báo cũ từ các năm trước có khả năng truy cập và thu thập bị hạn chế hơn rất nhiều, có thể là do cấu trúc website thay đổi theo thời gian hoặc là do trang lưu trữ dữ liệu các bài báo cũ không còn tồn tại nữa. Do đó, dữ liệu được giữ theo số lượng thực tế thay vì cố gắng thu thập phân bố đồng đều giữa các năm.

Ngoài ra, sẽ có vấn đề là nếu muốn dữ liệu Human là viết hoàn toàn bởi con người, ít ảnh hưởng bởi AI nhất thì tại sao không thu thập dữ liệu từ các năm trước trở xuống, ví dụ như năm 2023, là thời điểm AI chưa phát triển mạnh để các tác giả sử dụng để hỗ trợ khả năng viết bài của họ. Lý do là vì nếu ta chỉ thu thập từ các năm trước, giả sử như năm 2023 trở về trước, thì dữ liệu sẽ bị hạn chế về số lượng và tính đa dạng của nó, đó là yếu tố sự chuyển dịch theo thời gian, giả sử như năm 2023, các bài viết chưa có đề cập đến các chủ đề như Claude, OpenClaw, Gwen, ... Trong khi các bài báo hiện nay thì lại rất nhiều, điều đó vô tình khiến một vấn đề lớn là khi sau này hệ thống mà triển khai thì khi mà đưa một bài báo mà mô hình chưa thấy chủ đề đó bao giờ thì sẽ khó mà phân loại được, vì bản thân mô hình học sâu,

mô hình được sử dụng trong đồ án ngoài khả năng học cấu trúc câu, thì nó còn học thêm từ vựng, chủ đề và ngữ cảnh xuất hiện trong câu, do đó khả năng tổng quát hóa của mô hình sẽ trở nên kém hơn nếu đem đi triển khai thực tế. Do đó, đồ án quyết định xây dựng dữ liệu qua các năm, từ năm 2010 đến 2026 và tập trung hầu hết từ năm 2024 - 2026 để đảm bảo tính đa dạng về chủ đề, đa dạng hóa các nguồn và sát với dữ liệu thực tế để đi vận hành triển khai hệ thống.

3.2.3 Xây dựng dữ liệu AI

Dữ liệu AI được xây dựng dựa trên các mô hình ngôn ngữ lớn (LLM) nhằm tạo ra các mẫu bài báo, văn bản phục vụ cho viết huấn luyện mô hình phân biệt giữa văn bản do con người viết hay AI tạo ra. Dữ liệu bài báo AI được xây dựng bằng cách đưa ra chủ đề kết hợp với yêu cầu hoàn chỉnh để mô hình LLM tạo ra các bài báo, tức là sẽ chọn các chủ đề liên quan đến lĩnh vực đồ án, ví dụ như trí tuệ nhân tạo, smartphone, ứng dụng công nghệ, ... Sau đó, đưa ra các yêu cầu cụ thể về độ dài, cấu trúc bài viết, số lượng đoạn văn để đảm bảo giống như các bài báo từ con người, có nguồn rõ ràng để cho mô hình LLM sinh ra.

Ở phạm vi đồ án, mô hình LLM sử dụng là bao gồm Gemini và DeepSeek, vì lý do là khi sử dụng các mô hình khác, ví dụ như ChatGPT hay Claude thì sẽ tốn token và tốn rất nhiều chi phí mà ngân sách thì hạn hẹp cho nên lựa chọn hai mô hình đó để sinh bài báo, văn bản là cách tối ưu cho phạm vi đồ án. Tuy vậy, vẫn có một nhược điểm đó là nếu giới hạn các mô hình sinh văn bản như vậy thì khi mà triển khai thì người dùng vẫn có thể sử dụng các mô hình ngôn ngữ khác tốt hơn để sinh bài báo, ví dụ như các mô hình Opus 5 hay Fable 5 của Claude với khả năng viết lách giống như con người hơn thì hệ thống sẽ vẫn khó để phân biệt loại bài báo đó là do con người hay AI tạo nên.

3.2.4 Xây dựng dữ liệu AI-rewritten

Dữ liệu AI-rewritten, chính là dữ liệu AI tạo ra dựa trên dữ liệu con người, tức là sử dụng AI để viết lại, thay đổi cấu trúc và văn phong dựa trên dữ liệu do con người, từ các nguồn uy tín. Mục đích của quá trình này là tạo ra các bài báo, văn bản mô phỏng trường hợp mà các bài viết ban đầu do con người viết nhưng được sử dụng AI để diễn đạt lại, và bài viết đó vẫn chứa chất xám của con người. Trong thực tế, có một số người lợi dụng các công cụ AI để viết lại những văn bản được tạo bởi con người nhằm làm cho nội dung khó bị phát hiện hơn, với mục đích sử dụng bài viết, chất xám của người khác để thay đổi cấu trúc, văn phong làm bài viết của mình.

Trong các trường hợp đó thì nội dung, thông tin chính vẫn còn được giữ lại, trong khi cách sử dụng từ ngữ, cấu trúc câu hay phong cách diễn đạt lại được thay đổi. Điều này vô tình dẫn đến một thách thức lớn đối với các hệ thống phát hiện văn bản AI, lý do đơn giản là bởi vì các văn bản do AI viết lại không còn mang những đặc điểm rõ ràng của các văn bản do AI tạo hoàn toàn từ đầu.

Để mô phỏng trường hợp đó thì trong đồ án, em đã xây dựng dữ liệu AI-rewritten dựa trên các bài báo, bài viết thuộc lớp Human đã thu thập. Quá trình tạo dữ liệu như sau, nội dung của bài báo từ lớp Human sẽ được đưa vào các mô hình LLM, cụ thể trong đồ án thì các mô hình LLM vẫn giống như cách xây dựng tập dữ liệu AI tự sinh hoàn toàn, đó là mô hình Gemini và Claude, kết hợp với yêu cầu (prompt) viết lại với cấu trúc câu thay đổi, độ dài văn

bản không thay đổi quá nhiều, thay đổi từ vựng và cách diễn đạt. Sau đó các bài báo được mô hình LLM sinh ra và sẽ được gán vào lớp AI với một thuộc tính phân biệt là "rewritten".

3.2.5 Cấu trúc và tổ chức dữ liệu

Dữ liệu được tổ chức dưới dạng bảng, mỗi hàng là một mẫu bài báo tương ứng với các đặc trưng liên quan đến quá trình phụ vụ huấn luyện mô hình. Tập dữ liệu (Dataset) được thiết kế nhằm phân biệt các bài báo do con người viết và do mô hình ngôn ngữ lớn góp phần tạo ra.

```
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   url              3750 non-null   str     
1   source           3750 non-null   str     
2   author           3750 non-null   str     
3   publish_date     3750 non-null   str     
4   text             5000 non-null   str     
5   original_text     1250 non-null   str     
6   generated_type    5000 non-null   str     
7   label            5000 non-null   int64   
dtypes: int64(1), str(7)
memory usage: 312.6 KB
```

Hình 15: Cấu trúc dữ liệu của hệ thống

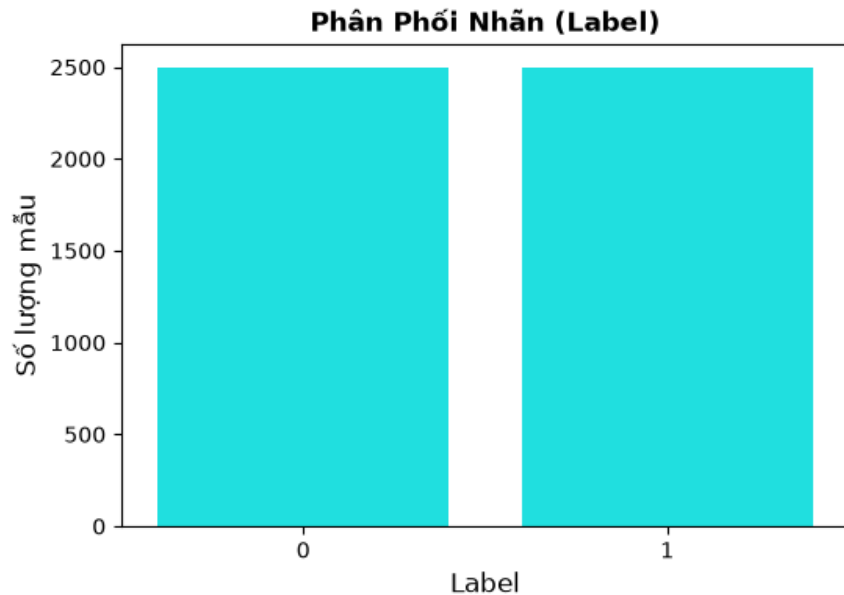
Trường	Loại	Ý nghĩa
url	Metadata	Nguồn URL của các bài báo
source	Metadata	Tên miền của các bài báo.
author	Metadata	Tên tác giả của bài báo.
publish_date	Metadata	Ngày xuất bản bài báo.
text	Đặc trưng đầu vào	Nội dung văn bản được sử dụng cho quá trình huấn luyện và dự đoán.
original_text	Dữ liệu tham chiếu	Văn bản gốc trước khi được mô hình ngôn ngữ viết lại.
generated_type	Metadata	Phương pháp tạo ra bài báo.
label	Nhãn	Nhãn phân loại, biểu thị văn bản Human hoặc AI.

Bảng 1: Mô tả các trường trong tập dữ liệu

Như vậy, bộ dữ liệu được tổ chức với trường thông tin được sử dụng để lưu trữ nội dung văn bản, nhãn phân loại và các thông tin liên quan đến nguồn dữ liệu. Ngoài ra, text là nội dung bài báo và label là thành phần chính để phục vụ cho quá trình mô hình học và dự đoán, còn các trường còn lại được sử dụng cho mục đích phân tích và khai thác sâu vào dữ liệu.

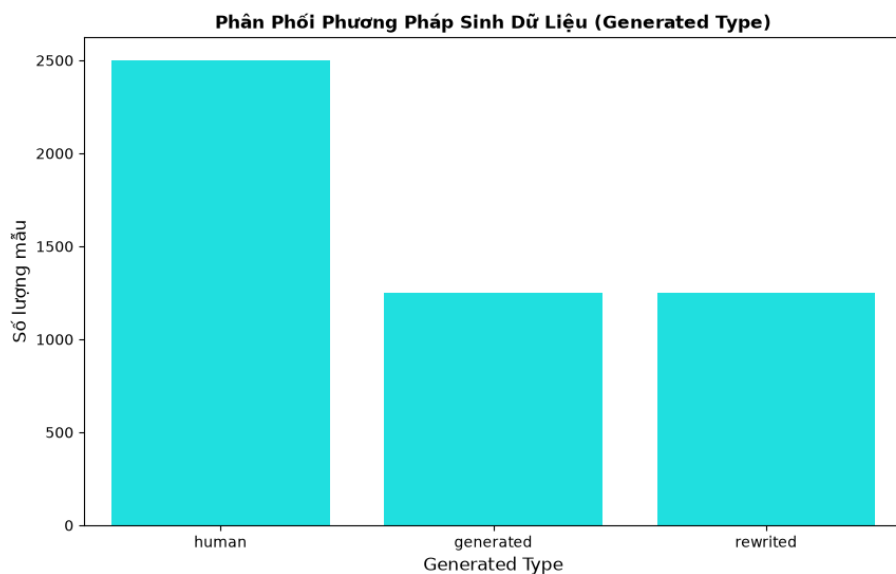
3.2.6 Phân bố dữ liệu

Như ta đã thấy ở Hình 15, dữ liệu là tổng cộng gồm có 5000 mẫu. Trong đó, cột label được chia thành hai lớp có số lượng mẫu bằng nhau, đó là lớp AI và Human có số lượng mẫu đều là 2500 mẫu.



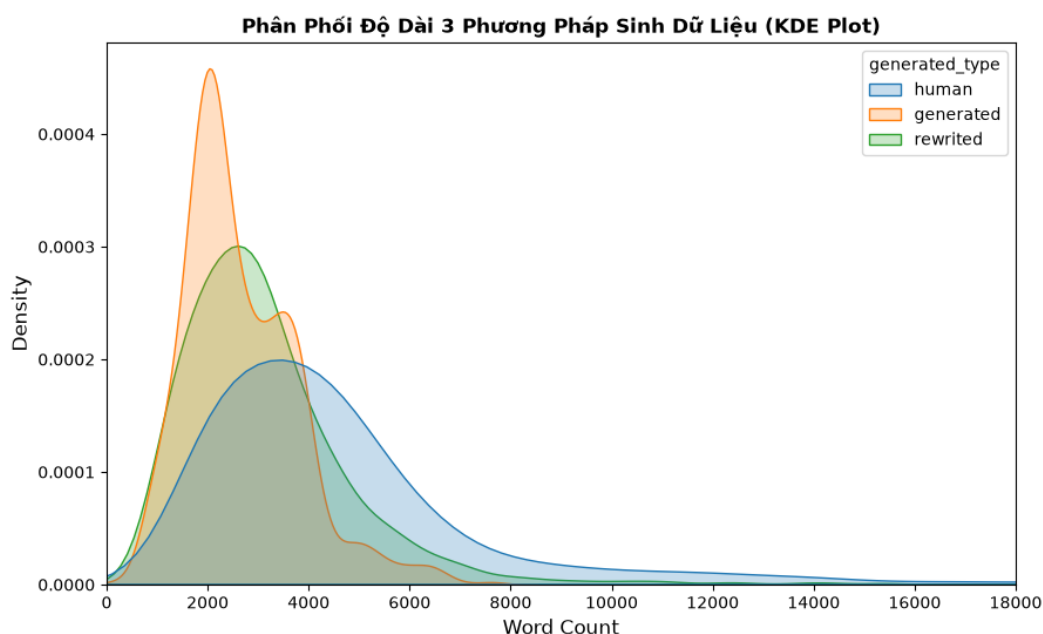
Hình 16: Phân bố nhãn label

Ngoài ra, trường generated_type cũng đã phân bố đều, với ba thành phần đó là Human, Generated (bài báo do AI sinh ra hoàn toàn từ đầu), Rewritten (bài báo do AI viết lại từ bài báo của con người, có nguồn rõ ràng). Dữ liệu cũng được tổ chức, phân bố đều với Human chiếm tới 2500 mẫu và hai thành phần còn lại, mỗi phần tương ứng 1250 mẫu.



Hình 17: Phân bố phương pháp tạo sinh dữ liệu

Bên cạnh sự phân bố nhãn và phương pháp sinh dữ liệu, độ dài văn bản cũng là một yếu tố, đặc điểm cần đáng được xem xét trong quá trình phân tích toàn bộ dữ liệu. Sự khác biệt độ dài giữa các nhóm văn bản có thể thể phản ánh những đặc điểm khác nhau trong quá trình biên soạn và tạo nội dung, đồng thời đánh giá được mức độ tương đồng về mặt độ dài giữa dữ liệu AI và Human.



Hình 18: Phân bố độ dài của các phương pháp tạo sinh dữ liệu

Biểu đồ ước lượng mật độ nhân (Kernel Density Estimate - KDE) là một loại biểu đồ được sử dụng để trực quan hóa sự phân phối của một tập dữ liệu liên tục. Ở hình 18 trên, biểu đồ thể hiện mật độ của phân phối số lượng từ dựa trên ba phương pháp sinh dữ liệu.

Ta có thể thấy nhóm sử dụng AI để tự sinh dữ liệu từ đầu (Generated) có mật độ cao nhất, nằm trong khoảng 2.000 - 3.500 từ và một số ít tập trung vào khoảng gần 4.000 - 4.500 từ ở mỗi bài. Điều này dữ liệu AI sinh văn bản từ đầu trong đề án có xu hướng tuân theo một khuôn mẫu về độ dài, ít biến động và phần lớn độ dài của văn bản khá là tương đồng.

Ngoài ra, nhóm Human có mật độ phân bố trải rộng rõ rệt hơn so với hai nhóm còn lại, khoảng 3.500 - 4.500 từ mỗi bài. Ở phía bên phải còn tồn tại rất nhiều văn bản trải rộng, tức là còn tồn tại nhiều văn bản có độ dài lớn hơn so với mặt bằng chung. Điều này phản ánh tính đa dạng của nhóm Human khi xây dựng các nội dung bài báo so với hai nhóm còn lại.

Tuy vậy, mật độ phân phối độ dài của ba nhóm có một phần chồng lấn, tập trung từ 2.000 - 4.000 từ khá nhiều. Cho thấy được độ dài cũng chưa phải là một đặc trưng đủ mạnh để mô hình đưa ra quyết định, còn phải phụ thuộc thêm ngữ nghĩa, cấu trúc câu và từ vựng của từng loại bài báo.

3.3 Tiền xử lý dữ liệu

3.3.1 Làm sạch và chuẩn hóa văn bản

Sau khi đã phân tích, tìm hiểu cấu trúc, sự phân bố dữ liệu thì ta sẽ thực hiện quá trình làm sạch dữ liệu. Trong đồ án, quá trình làm sạch dữ liệu gồm ba bước: loại bỏ HTML, URL và loại bỏ khoảng trắng đầu và cuối của chuỗi, lý do là vì các mã code hay đường dẫn thì nó không phản ánh cách viết của người hay AI và khoảng trắng chỉ làm tốn tài nguyên xử lý mà không mang lại giá trị ngữ nghĩa.

```
1 def clean_text(text):
2     text = BeautifulSoup(text, 'html.parser').get_text()
3     text = re.sub(r'http\S+|www\S+', '', text)
4     text = re.sub(r'\s+', ' ', text).strip()
5     return text
```

Đoạn mã 1: Hàm làm sạch dữ liệu

```
# Test case
text = " >>> Xem ngay tại: https://openai.com hoặc https://example.com/news?id=123 <p><b>Xin chào</b>, đây là <a href='#>một bài viết</a>.</p> "
cleaned_text = cleaner.clean_text(text)
cleaned_text

'>>> Xem ngay tại: hoặc Xin chào, đây là một bài viết.'
```

Hình 19: Đoạn văn bản sau khi đã làm sạch

Tuy nhiên vẫn có một điều cần làm rõ là trong đồ án phân loại văn bản AI này thì vẫn giữ nguyên dấu câu, chữ hoa, chữ thường, định dạng từ và các biểu cảm của văn bản gốc. Lý do là vì các thành phần đó là tín hiệu cực mạnh để phân biệt văn bản do người viết hay do AI tạo ra, các thành phần đó cung cấp thông tin về phong cách viết, cấu trúc câu và cách sử dụng ngôn ngữ cho mô hình phân loại. Nhưng em vẫn không thể khẳng định đây là những đặc điểm chính xác luôn có khả năng phân biệt giữa con người viết hay do AI tạo ra. Bởi vì một tác giả họ có thể có khả năng viết một bài báo rất mạch lạc, tuân thủ nguyên tắc và sử dụng dấu câu một cách chính xác, trong khi các mô hình LLM hiện nay cũng có khả năng tạo ra văn bản giống như con người. Vì vậy, đồ án đã không loại bỏ những thành phần này mà giữ nguyên chúng để mô hình có thể tự học mức độ đóng góp của từng đặc điểm dựa trên dữ liệu huấn luyện, thay vì áp đặt một đặc điểm nào đó chắc chắn là của Human hay là AI, tức là ví dụ như văn bản có nhiều dấu câu thì mình dự đoán đó là AI hay văn bản có lỗi chính tả là của Human thì các giả định này không phải lúc nào cũng đúng.

3.3.2 Tokenization

3.4 Xây dựng và fine-tuning mô hình BamiBERT

Phân tích các trường hợp nhầm Human thành AI, nhầm AI thành Human, ảnh hưởng của độ dài văn bản và sự khác biệt giữa generated với rewritten.

3.5 Xây dựng chức năng giải thích

3.5.1 Quy trình Integrated Gradients

Mô tả baseline padding, target AI, attribution trên embedding và cách gộp subword thành từ.

3.6 Tích hợp Gemini

Giải thích quy ước màu, độ đậm và cách hiển thị các từ có ảnh hưởng đến dự đoán.

3.7 Xây dựng giao diện và API

Trình bày Infidelity, Sensitivity và kết quả theo từng nhóm dữ liệu. Cần lưu ý attribution không phải là chứng minh quan hệ nhân quả.

Chương 4

XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1 Kiến trúc tổng thể

Trình bày mối quan hệ giữa React frontend, FastAPI backend, ModelService, module Captum, Gemini và database SQLite.

4.2 Xây dựng backend

4.2.1 Khởi tạo và quản lý mô hình

Mô tả việc tải checkpoint và load model một lần trong lifecycle của FastAPI.

4.2.2 Các API chính

Trình bày các endpoint /health, /predict, /explain, /explain-llm, nhóm API xác thực và nhóm API lịch sử dự đoán.

4.2.3 Xử lý giải thích

Mô tả luồng gọi ModelService, Captum và Gemini; giới hạn độ dài đầu vào, cache và rate limiting.

4.3 Xây dựng frontend

4.3.1 Chức năng phân tích văn bản

Mô tả màn hình nhập văn bản, gửi request và hiển thị xác suất Human/AI bằng gauge.

4.3.2 Hiển thị attribution

Mô tả highlight theo từ, màu sắc và cường độ ảnh hưởng.

4.3.3 Hiện thị giải thích ngôn ngữ tự nhiên

Mô tả các bullet giải thích do Gemini sinh dựa trên bằng chứng từ mô hình.

4.4 Xác thực người dùng và lịch sử

Trình bày đăng ký, đăng nhập, JWT, bcrypt, lưu lịch sử dự đoán, xem chi tiết, xóa lịch sử và upload avatar.

4.5 Đóng gói và triển khai

Mô tả Docker Compose, backend CPU inference, frontend build bằng Vite và phục vụ bằng Nginx.

4.6 Kiểm thử hệ thống

Đề xuất các kịch bản kiểm thử API, xác thực, dự đoán, explainability, Gemini và quy trình người dùng trên giao diện web.

4.7 Hạn chế triển khai

Nêu các điểm cần hoàn thiện: giới hạn 512 token, thiếu sliding window, thiếu test tự động đầy đủ, cần thống nhất cấu hình notebook/production và kiểm tra schema database trước khi chuyển sang hệ quản trị khác.

4.8 Kết luận chương

Đánh giá khả năng chuyển mô hình nghiên cứu thành một ứng dụng có thể sử dụng qua web.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả đạt được

Tóm tắt pipeline dữ liệu, mô hình BamiBERT, hệ thống giải thích và ứng dụng web.

Hạn chế

Nêu các hạn chế như dữ liệu Human dạng proxy label, kết quả thấp hơn trên AI-rewritten, giới hạn 512 token và thiếu bộ kiểm thử tự động đầy đủ.

Hướng phát triển

Đề xuất human annotation, sliding-window inference, mở rộng dữ liệu, calibration xác suất và bổ sung kiểm thử tự động.

TÀI LIỆU THAM KHẢO

1. Tài liệu BaniBERT và mô hình BERT tiếng Việt.
2. Tài liệu PyTorch và Hugging Face Transformers.
3. Tài liệu Captum về Integrated Gradients.
4. Tài liệu FastAPI, React và Docker.